KristaTech (KRISTA) Teknik Whitepaper

Proof-of-BLS (PoBLS) Önerici Seçimi, Korum Dirençli İşbirlikçi Hibrid Mutabakat ve JSON Tabanlı Bildirimsel Akıllı Sözleşmeler (MESCAL) ile Güçlendirilmiş Blokzincir Protokolü

Özet
Bu çalışma, mutabakat merkeziyeti, blok liderlerini hedef alan Hizmet Dışı Bırakma (DoS) saldırıları ve geleneksel sanal makinelerin mimari karmaşıklıklarını çözmek üzere tasarlanan KristaTech (KRISTA) blokzincir protokolünü tanıtmaktadır. KristaTech, onaylayıcı kümesinin seçimini blok teklif etme (proposing) sürecinden ayıran iki katmanlı bir konsensüs mimarisi uygulamaktadır. Seçim katmanı olan **ADAM (A Decentralized Approach Model)**, Doğrulanabilir Rastgele Fonksiyon (VRF) tabanlı döngüsel tohumlar kullanarak blok yüksekliği başına N adet onaylayıcı (madenci) ve 1 adet koordinatörü deterministik olarak seçer. Blok önerme katmanı olan **Proof of BLS (PoBLS)** ise seçilen bu onaylayıcı havuzu içinden geçici (ephemeral) kriptografik BLS anahtarları yardımıyla XOR mesafe hesabı yaparak blok üreticisini dinamik bir piyango mekanizmasıyla belirler. Bu sayede ağ, ön hesaplama (pre-computation) ve lider hedefli DoS saldırılarına karşı tam koruma sağlar. Çevrimdışı onaylayıcılar veya ağ bölünmeleri durumunda liveness (canlılık) durumunun korunabilmesi için, dinamik bir eşik değeriyle doğrulanan boş vektör yer tutucu (`std::vector<unsigned char>()`) mekanizması geliştirilmiştir. Akıllı sözleşmeler, Bitcoin benzeri yığın tabanlı `CScript` bayt koduna doğrudan derlenen bildirimsel ve sade JSON yapısındaki **MESCAL (Minimalistically Envisioned Smart Contract Assembling Language)** dili ile yürütülür. Bu yapı, durum tabanlı zafiyetleri ve gaz ücreti hesaplama karmaşıklıklarını tamamen ortadan kaldırır. Son olarak, ağın sürdürülebilirliği, **%1.9 üç aylık (~90 günlük) emisyon azalması (decay)** ve **Model D İşbirlikçi Paylaşım** modeli ile yönetilen **210 Milyon KRISTA** üst sınırı (hard cap) ile güvence altına alınmıştır. Bu ekonomik model; net blok ödüllerini pasif masternode'lar (%35), aktif LLMQ korum üyeleri (%10), blok üreticisi (%15) ve katılımcı onaylayıcılar (%40: 7 Masternode onaylayıcı koltuğu için %20 ve 4 bağımsız madenci koltuğu için %20) arasında adil bir şekilde dağıtırken, ekosistemin geliştirilmesi amacıyla kurumsal geliştirici ve musluk (faucet) fonlarını da içermektedir.

Anahtar Kelimeler: Mutabakat Protokolleri, Blokzincir Güvenliği, Kriptografik Piyango, Bildirimsel Akıllı Sözleşmeler, Tokenomi.

---

1. Giriş ve Arka Plan

Dağıtık mutabakat protokolleri, özünde Bizans Generalleri Problemini hasmane ve açık ağ ortamlarında çözmeyi amaçlar. Geleneksel İş Kanıtı (PoW) ve Pay Kanıtı (PoS) tasarımları ağ koordinasyonunu başarıyla sağlamış olsalar da, beraberlerinde kritik yapısal zayıflıklar getirmektedir:

1. **Konsensüs Merkeziyeti**: PoW ağlarında ölçek ekonomisi, hash gücünün sınırlı sayıda endüstriyel madencilik havuzunda toplanmasına yol açar. PoS ağlarında ise zenginlik birikimi ("zenginin daha da zenginleştiği" dinamikler), yüksek miktarda teminat tutan cüzdanların blok üretimini tekeline almasına neden olur.
2. **Lider Hedeflenebilirliği**: Bir sonraki blok üreticisinin önceden bilindiği veya tahmin edilebildiği protokollerde, saldırganlar bu düğümü hedef alan DDoS saldırıları düzenleyebilir veya işlemleri sansürlemesi için blok üreticisine baskı uygulayabilir.

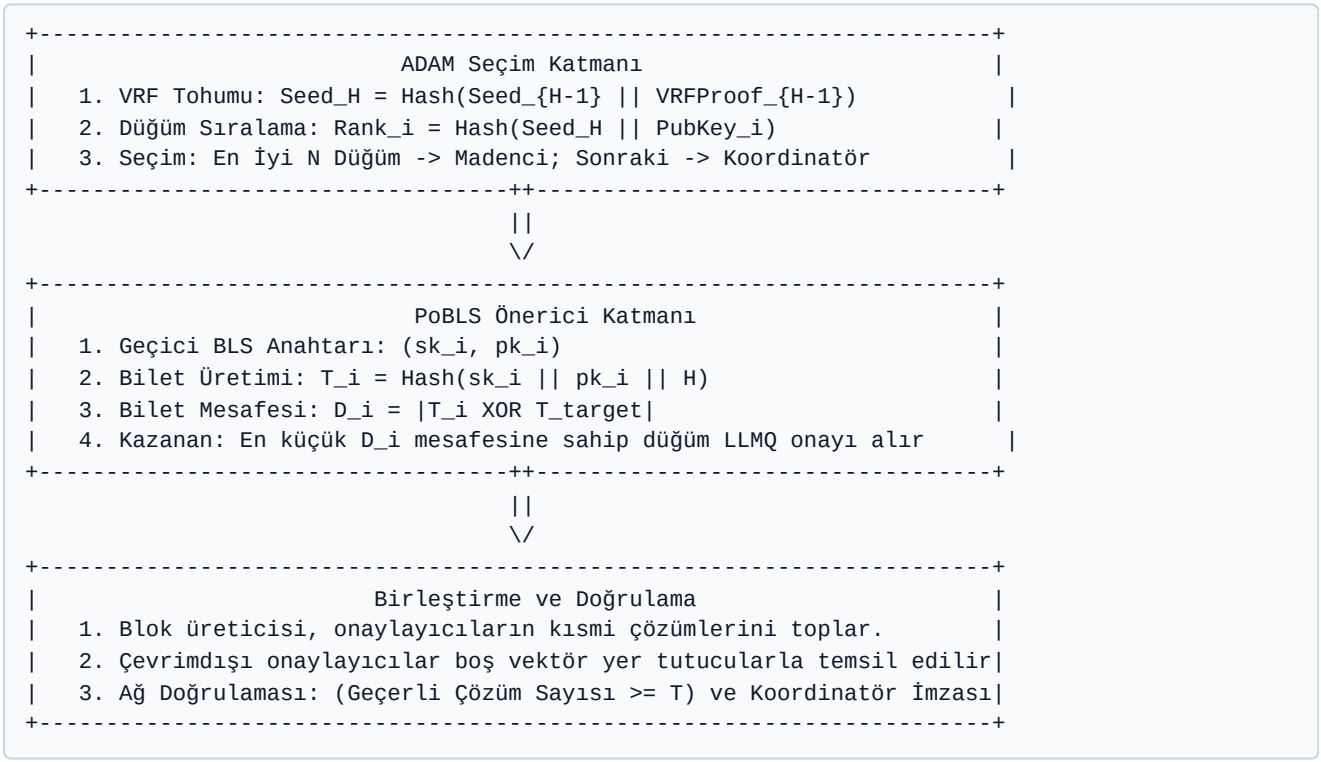
3. **Blok Grinding (Öğütme) Saldırıları**: Blok başlığındaki veya işlemlerdeki verilerin değiştirilmesiyle sonraki lider seçimlerinde kullanılacak rastgelelik kaynağının manipüle edilebildiği sistemlerde, kötü niyetli aktörler blok öğüterek kendi seçilme olasılıklarını artırabilir.
4. **Akıllı Sözleşme Sadeliği**: Geleneksel sanal makineler, durum yönetimindeki karmaşıklık nedeniyle işlemlerde öngörülemeyen hatalara yol açabilir. Güvenli geçişleri garanti altına almak için daha sade ve öngörülebilir bir akıllı sözleşme dili gereklidir.

KristaTech, bu sorunları çözmek amacıyla iki katmanlı işbirlikçi bir konsensüs mimarisi sunar. Blok üretim sürecini deterministik bir seçim katmanı (ADAM) ve öngörülemez bir önerme katmanı (PoBLS) olarak ikiye bölerek hem Sybil saldırılarına karşı direnç hem de DDoS koruması elde eder. Akıllı sözleşme yürütümü, sade ve deterministik çalışan **MESCAL** dili ile sınırlandırılarak blokzincir durum güvenliği korunur. Ekosistem, uzun vadeli sürdürülebilirliği destekleyen emisyon planıyla güvence altına alınmıştır.

---

2. Konsensüs Mimarisi

KristaTech'in merkezinde işbirlikçi bir mutabakat süreci yer alır. Blok yaşam döngüsü **Seçim Aşaması**, **Çözüm Aşaması** ve **Birleştirme Aşaması** olmak üzere üç aşamadan oluşur.



2.1. ADAM (A Decentralized Approach Model)

ADAM, ağın kimlik ve seçim katmanıdır. Tüm düğümlerin serbestçe yarışmasına izin vermek yerine (ki bu durum Sybil zafiyetlerine yol açar), her blok yüksekliği için deterministik olarak N adet Madenci (Validator) ve 1 adet Koordinatör seçer.

2.1.1. VRF Döngüsel Tohumları

Blok öğütme (grinding) saldırılarını engellemek amacıyla, seçim algoritmasında kullanılan rastgelelik Doğrulanabilir Rastgele Fonksiyon (VRF) ile üretilir. H blok yüksekliğindeki seçim tohumu şu şekilde formüle edilir:

$$\text{Seed}_H = \text{Hash}(\text{Seed}_{H-1} \| \text{VRFProof}_{H-1})$$

Burada $H-1$ yüksekliğindeki Koordinatörün ürettiği VRF kanıtı (VRFProof), $H-2$ yüksekliğindeki tohum üzerine hibrit BLS12-381 + ECDSA fallback imza mekanizması (`SignBLSWithECDSAFallback`) kullanılarak atılan deterministik imzadır. Ortaya çıkan BLS imzası (BLS genel anahtarının bir ECDSA imzasıyla yetkilendirilmesiyle doğrulanır) tamamen deterministik olduğu için, Koordinatör imza değerini manipüle ederek $H+1$ yüksekliğindeki seçimleri kendi lehine değiştiremez. Bu döngüsel tohum blok başlığında saklanır, böylece tarihsel rastgelelik verileri geriye dönük olarak denetlenebilir hale gelir.

2.1.2. Düğüm Seçimi, Sıralama ve Çift Kulvarlı (Dual-Track) Kotalar

Aktif aday havuzundaki (P) her düğüm i için benzersiz bir puan sıralaması hesaplanır:

$$\text{Rank}_i = \text{Hash}(\text{Seed}_H \| \text{PubKey}_i)$$

Mainnet ve Testnet üzerinde seçim havuzu, aktif Masternode'lardan (harcanmamış 4.200 KRISTA teminata ve son 2.880 blok içinde zincir üstü heartbeat ping'e sahip olanlar) ve aktif kayıtlı madencilerden (`250 KRISTA` Coin-Lock, `10 KRISTA` fonlu PoW-Lock veya `0 KRISTA` Zero-Coin PoW ile kayıt olanlar) dinamik olarak oluşturulur. `nHeight >= consensus.nAdamDualTrackHeight` (`UPGRADE_ADAM` = Mainnet/Testnet'te blok 200, Regtest'te blok 1) olduğunda, düşük maliyetli PoW kayıtlarının yüksek teminatlı düğümleri dışlamasını önlemek ve aynı zamanda izinsiz açık madencilik katılımını garanti altına almak amacıyla $N = 11$ **Madenci (Validator)** koltuğu **Çift Kulvarlı Kota Seçimi (`ElectDualTrackValidators`)** ile belirlenir:

- **1. Kulvar (Teminatlı Havuz — Toplam 8 Koltuk)**:
  - **Alt-Kulvar 1A (Aktif Masternode'lar — 7 Koltuk)**: `4.200 KRISTA` teminatlı aktif Masternode'lar için ayrılmıştır (`static_assert(ADAM_MASTERNODE_TRACK_QUOTA >= 7)` ile aktif Masternode'ların bootstrap sonrası `7` eşik değerini tek başına karşılaması garanti edilir).
  - **Alt-Kulvar 1B (Coin-Lock Doğrulayıcılar — 1 Koltuk)**: `250 KRISTA` Coin-Lock madencileri için ayrılmıştır; Alt-Kulvar 1A ve 1B arasında Kulvar-1 içi doğrudan taşma (spillover) uygulanır.
- **2. Kulvar (Açık PoW Havuzu — Toplam 3 Koltuk)**: Açık PoW madencilerine (`10 KRISTA PoW-Lock`, `0 KRISTA Zero-Coin PoW` ve `nAdamBootstrapLimit` altındaki bootstrap anahtarları) ayrılmıştır; kulvarlardan birinde yeterli aday bulunmadığında kulvarlar arası çift yönlü taşma uygulanır.

Başlangıç aşamasında (bootstrap) düğümlerin Masternode kurulumu için gereken 4.200 KRISTA teminat biriktirmesini kolaylaştırmak amacıyla, Mainnet üzerinde 2.200. blok yüksekliğinin (Model D aktivasyonu) altında Coin-Lock kaydı devre dışı bırakılmış ve düğümlerin PoW-Lock (`10 KRISTA` kilitli veya `0 KRISTA` Zero-Coin PoW) kullanması sağlanarak blok ödüllerinin likit kalması temin edilmiştir. Regtest üzerinde ise, otomatik testleri kolaylaştırmak amacıyla havuz otomatik olarak 15 adet genel anahtarı içerir.

2.1.3. Mod Dinamikleri ve Deterministik Blok Yüksekliği Aktivasyonu

Ağın sorunsuz bir şekilde başlatılabilmesi (bootstrapping) için ADAM iki farklı modda çalışabilir:

- **Fallback Modu (Sürüm 11)**: Ağın ilk başlangıç (bootstrap) aşamasında (`UPGRADE_ADAM` $\leq H <$ `UPGRADE_POMBL`) çalışır. Aktif Masternode sayısı yeterli eşik değerinin altında olduğundan, onaylayıcı havuzu tamamen aktif kayıtlı madencilerden oluşturulur. Madenci sayısı $N \in [11, 14]$ arasında dinamik olarak değişir ve gereken asgari geçerli çözüm eşiği T, `GetAdamThreshold(nHeight)` fonksiyonu tarafından yönetilir (`nHeight < nAdamThresholdBootstrapLimit` , yani Mainnet'te 50.000, Testnet'te 100.000, Regtest'te 1.000. bloğun altında ağ canlılığını korumak için 5, sonrasında ise `nAdamThreshold` parametresi uyarınca Mainnet/Regtest'te 7 ve Testnet'te 3 geçerli çözüm).
- **Standart Mod (Sürüm 12)**: 3-permütasyonlu bulmaca özetleme zinciri ile tam kooperatif Çoklu Kanıt Algoritması (PoMBL) konsensüsünü etkinleştirir. Madenci sayısı N sabit olarak `nAdamMinersCount` (11), asgari geçerli çözüm eşiği T ise `GetAdamThreshold(nHeight)` fonksiyonu ile belirlenir (bootstrap süresince Mainnet'te 50.000, Testnet'te 100.000, Regtest'te 1.000. bloğun altında 5, sonrasında Mainnet/Regtest'te 7; Testnet'te 3). Aktif Masternode sayısı eşik değerini aştığında Koordinatör aktif Masternode listesinden dinamik olarak seçilirken, 11 komite doğrulayıcısı `ElectDualTrackValidators` ile seçilir.
- **Etkinleştirme (`Consensus::UPGRADE_POMBL`)**: Fallback Modu (Sürüm 11) ile Standart Mod (Sürüm 12) arasındaki geçiş, idari Spork anahtarlarına bağımlı olmaksızın doğrudan `Consensus::UPGRADE_POMBL` blok yüksekliği (`nPoMBLHeight` = Mainnet'te **2.000**, Testnet'te **400** ve Regtest'te **300**) ile deterministik olarak denetlenir; bu yükseklik ve sonrasında `nVersion < 12` olan bloklar `bad-version` hatası ile kesin olarak reddedilir.

2.1.4. Bootstrap Güvenliği ve Başlangıç Zorluğu

Madenciliğin 1. bloktan itibaren halka açık olacağı ve birden fazla düğümün aynı anda kazı yapacağı ana ağın (mainnet) ilk açılış aşamasını (bootstrap) güvenli hale getirmek için, başlangıç PoW zorluk sınırı (`powLimit`) `~UINT256_ZERO >> 20` (genesis bloğunun `0x1e0ffff0` değerindeki `nBits` parametresine eşit olacak şekilde) olarak sıkılaştırılmıştır. Bu sayede ilk 199 bloğun mikrosaniyeler içinde aşırı hızlı kazılması engellenerek düğümlerin birbirleriyle kararlı P2P bağlantıları kurması ve blokları ağda sağlıklı bir şekilde yayması için yeterli zaman kazanılır. Bu durum, düğümlerin 199. blokta farklı hash değerlerine sahip olarak bölünmesi (fork split) ve 200. blokta korum (quorum) oluşturamayarak kilitlenmesi (deadlock) riskini tamamen ortadan kaldırır.

2.2. Proof of BLS (PoBLS) Konsensüsü

ADAM katmanı tarafından belirlenen onaylayıcı kümesi içinden blok önericisinin seçimi, **Proof of BLS (PoBLS)** piyangosu ile gerçekleştirilir. Bu yapı önericinin kimliğini gizli tutarak hedefli DoS saldırılarını imkansız kılar.

2.2.1. Geçici Bilet Üretimi

Elected durumundaki her onaylayıcı i, H yüksekliği için geçici bir BLS anahtar çifti (sk_i, pk_i) üretir. Bu anahtarlar kullanılarak bir piyango bileti T_i hesaplanır:

$$T_i = \text{Hash}(sk_i \parallel pk_i \parallel H)$$

Onaylayıcı, özel anahtarı ifşa etmeden sahipliğini kanıtlamak için önceki blok özetini ($Hash_{\text{prev}}$) geçici özel anahtarıyla imzalar:

$$\sigma_i = \text{Sign}_{sk_i}(Hash_{\text{prev}})$$

Bu bilgilerden oluşan katılım mesajı (pk_i, σ_i, T_i) ağdaki aktif **Long-Living Masternode Quorum (LLMQ)** yapısına iletilir. Her LLMQ tam olarak 5 üyeden oluşur. Model D aktif olduğunda, `CQuorumSignature::Verify()` (`src/llmq.cpp`) içindeki korum imzası doğrulama eşiği katı bir BFT salt çoğunluğudur (`quorum.members.size() / 2 + 1`) ve tüm ağlarda (Mainnet, Testnet ve Regtest) 5 üyeli bir korum için en az **5'te 3** imzanın mevcut olmasını şart koşar; Mainnet ve Testnet üzerinde Model D aktivasyonu öncesinde ise bootstrap sürecini kolaylaştırmak için korum imzası doğrulaması atlanır (0 imza şartı). Ayrıca Mainnet, Testnet veya Regtest üzerinde, testlerin izole edilmesi amacıyla herhangi bir sabit kodlanmış (hardcoded) anahtar kimliği filtrelemesi veya yerel cüzdan kısıtlaması uygulanmaz, bu da tamamen merkeziyetsiz ve güven gerektirmeyen (trustless) bir test ortamı sağlar.

2.2.2. XOR Mesafesi ile Kazananın Belirlenmesi

Aktif döngüsel tohumdan (Seed_H) bir hedef özet (T_{target}) türetilir. LLMQ korumu, iletilen her bilet ile hedef arasındaki XOR mesafesini hesaplar:

$$D_i = |T_i \oplus T_{\text{target}}|$$

En küçük XOR mesafesine (D_i) sahip olan onaylayıcı, bloğu önerme hakkını kazanır. Geçici BLS anahtarları, grinding saldırılarını önlemek amacıyla düğümün kalıcı kimlik anahtarından türetilmelidir:

$$sk_i = \text{DeriveKey}(sk_{\text{node}}, Hash_{\text{prev}})$$

Bu kural, her onaylayıcının blok başına yalnızca tek bir bilet üretebilmesini sağlayarak bilet ön hesaplama yarışını engeller. Yeni aktif korumları seçmek için DKG oturumları Mainnet'te 1.400. bloğa kadar her 100 blokta bir, Testnet'te 1.000. bloğa / Regtest'te 800. bloğa kadar her 10 blokta bir çalıştırılır ve bu yüksekliklerden sonra her blokta (`nInterval = 1`) döndürülür.

2.2.3. Metrik Uzay ve XOR Mesafe Analizi

XOR işlemi ($\oplus$), $L = 256$ uzunluğundaki ikili anahtarlar kümesi üzerinde $d(x, y) = x \oplus y$ şeklinde bir metrik uzay (X, d) tanımlar. Bu metrik, bir metrik uzayın üç temel özelliğini karşılar:

1. **Ayırt Edilemezlerin Özdeşliği**: $d(x, y) = 0 \Leftrightarrow x \oplus y = 0 \Leftrightarrow x = y$
2. **Simetri**: $d(x, y) = x \oplus y = y \oplus x = d(y, x)$
3. **Üçgen Eşitsizliği**: $d(x, z) \leq d(x, y) \oplus d(y, z)$ ki bu durum XOR uzayında daha güçlü bir ultra-metrik özelliği sağlar:

   $$d(x, z) \leq \max(d(x, y), d(y, z))$$

Hedef tohum T_{target} sözde rastgele ve düzgün dağılımlı olduğu, biletler T_i de kriptografik olarak üretildiği için, elde edilen XOR mesafeleri D_i, $[0, 2^{256} - 1]$ aralığında bağımsız ve düzgün dağılımlı rastgele değişkenler olarak davranır. Bu sayede her onaylayıcının bloğu önerme olasılığı eşit derecede adil ($1/N$) dağılır.

2.2.4. Üretim (Production) Kod Uygulaması (`src/adam.cpp`, `src/llmq.cpp`, `src/crypto/bls.cpp`)

Üretim C++ konsensüs motoru, kavramsal PoBLS + ADAM matematiksel modelini zincir üstü deterministik yürütümle birebir birleştirir:

- **Deterministik ve Sabitlenmiş (Pinned) BLS12-381 Anahtar Türetimi (`DeriveBLSFromCKey` / `PinBLSPubKey`)**: Her onaylayıcı, koordinatör ve korum üyesi, düğüm anahtarından deterministik bir BLS12-381 anahtar çifti sk_i = BLST_KeyGen(SHA256D(sk_{node})), $pk_i = g_1^{sk_i}$ türetir ve bu anahtarı `CBLSSignedData` (`SignBLSWithECDSAFallback` / `VerifyBLSWithECDSAFallback`) aracılığıyla kalıcı `secp256k1` kimlik anahtarıyla yetkilendirir. İlk geçerli zincir üstü katılımda `(secp256k1 pubKey -> blsPubKey)` eşleşmesi LevelDB üzerinde kalıcı olarak sabitlenir (`PinBLSPubKey`); bu sayede anahtar öğütme (key grinding) ve blok içi VRF/bilet manipülasyonu matematiksel olarak imkansız hale gelir.
- **PoBLS Bileti (T_i), Hedef (T_{target}) ve XOR Metrik Mesafesi ($D_i = |T_i \oplus T_{\text{target}}|$)**: Seçilen her onaylayıcının imzalı kısmi çözümü (pk_i, σ_i) çiftini barındırır; `ComputePoBLSTicket` ve `ExtractPoBLSTicketFromSolution` fonksiyonları bu veriden T_i = SHA256D($pk_i \parallel \sigma_i \parallel H$) biletini üretir. T_{target} = SHA256D(Seed$_H \parallel H$) (`ComputePoBLSTarget`) hedefine karşı `ComputePoBLSXORDistance` fonksiyonu $(\{0,1\}^{256}, \oplus)$ uzayında $D_i = |T_i \oplus T_{\text{target}}|$ XOR mesafesini hesaplar ve `RankValidatorsByPoBLSDistance` aktif bilet sahibi onaylayıcıları en küçük XOR mesafesine göre sıralar.
- **5 Üyeli BFT LLMQ ve BLS12-381 İmza Agregasyonu (`ElectQuorumMembers`, `CQuorumSignature::Verify`, `CQuorumSignature::AggregateBLSSignatures`)**: Her H yüksekliği için harcanmamış 4.200 KRISTA teminatına (`pcoinsTip->HaveCoin`) sahip aktif Masternode'lar SHA256D(Seed ‖ collateralOutpoint) ile deterministik olarak seçilir. Korum üyeleri `blockHash` özetini (`PROPOSEBLOCK` / `QUORUMSIGSHARE`) deterministik BLS12-381 anahtarlarıyla (`DeriveBLSFromCKey`) imzalar. `CQuorumSignature::Verify` katı **5'te 3** BFT çoğunluğunu denetler ve imzalayan üyelerin BLS12-381 açık anahtarlarını (`CBLSPubKey::Aggregate`) ile imzalarını (`CBLSSignature::Aggregate`) birleştirerek $e(g_1, \Sigma_{\text{agg}}) = e(PK_{\text{agg}}, H_2(\text{blockHash}))$ eşleşme (pairing) denklemini doğrular.
- **Konsensüs Sinerjisi**:
  - **Sybil Koruması (ADAM)**: Katılım yalnızca teminatlı Masternode'lar ve Coin-Lock / PoW-Lock kayıtlı madencilerle sınırlandırılmıştır.
  - **DDoS Direnci (PoBLS XOR Sıralaması + Zaman Tabanlı Rotasyon)**: Birincil Koordinatör 45 saniye (`ADAM_FALLBACK_COORDINATOR_TIMEOUT_SECS = 45`) içinde blok yayınlamazsa, blok önerme yetkisi her 30 saniyede bir en küçük PoBLS XOR mesafesine ($D_i = |T_i \oplus T_{\text{target}}|$) göre sıralanmış (`RankValidatorsByPoBLSDistance`) 11 onaylayıcı arasında deterministik olarak döner.

2.3. Blok Başlığı Uzantıları ve Hashing Zinciri

ADAM güncellemesi aktif olduğunda, blok başlığı yapısı konsensüs kanıtlarını saklayacak şekilde genişletilir:

Alan	Tür	Açıklama
`vAdamMiners`	`std::vector<CPubKey>`	Seçilen onaylayıcılerin genel anahtarları.
`vAdamSolutions`	`std::vector<std::vector<char>>`	Kısmi çözümler (nonce + imza).
`vAdamVRFProof`	`std::vector<unsigned char>`	Koordinatörün önceki tohuma ait VRF imzası.
`vAdamCoordinatorSig`	`std::vector<unsigned char>`	Koordinatörün nihai blok özetine ait imzası.

2.3.1. Durumsuz Hashing Zinciri

Blok özeti hesaplanırken, blok başlığı seçilen onaylayıcıların sırasına göre ardışık bir hashing zincirinden geçirilir. Her onaylayıcı i için (zincir uzunluğu M = size($vAdamMiners$)):

1. Önceki blok özeti ve indeks bilgi kullanılarak raunda özel bir özet üretilir:

   roundHash_i = Hash(hashPrevBlock‖i)

2. İlk bayt v_i = roundHash_i[0] değeri alınarak tek bir aralarında asal çarpan hesaplanır:

   $m_i = v_i \mid 1$ (eğer $m_i < 3$ ise $m_i = 3$)

3. Hashing raundu yürütülür:

   - **Fallback Modu (Sürüm 11)**:
     - **Raund 0**: H_0 = CalculateAdamPuzzleHash(algo_0, SerializedHeader) $\times\, m_0 \pmod{2^{256}}$
     - **Raund i > 0**: H_i = CalculateAdamPuzzleHash(algo_i, H_{i-1}) $\times\, m_i \pmod{2^{256}}$ Burada algoritma indeksi şu şekilde hesaplanır:

     algoIndex = Hash(Seed_H‖MinerPubKey_i) (mod 18)

   - **Standart Mod (Sürüm 12)**:
     - **Raund 0**:
       - 0. madenci için algo1, algo2 ve algo3 algoritmalarını GetAdam3PermutationAlgos ile türetin.
       - Hesaplayın:

         $H_0^{(3)}$ = CalculateAdamPuzzleHash(algo3, SerializedHeader)

         $H_0^{(2)}$ = CalculateAdamPuzzleHash(algo2, $(H_0^{(3)} \times 1) \pmod{2^{256}}$)

         H_0 = CalculateAdamPuzzleHash(algo1, $(H_0^{(2)} \times 1) \pmod{2^{256}}$)

       - Çarpanı uygulayın:

         $H_{\text{prev}} = (H_0 \times m_0) \pmod{2^{256}}$

     - **Raund i** > 0:
       - i. madenci için algo1, algo2 ve algo3 algoritmalarını türetin.
       - Hesaplayın:

         $H_i^{(3)}$ = CalculateAdamPuzzleHash(algo3, H_{prev})

         $H_i^{(2)}$ = CalculateAdamPuzzleHash(algo2, $(H_i^{(3)} \times (i + 1)) \pmod{2^{256}}$)

         H_i = CalculateAdamPuzzleHash(algo1, $(H_i^{(2)} \times (i + 1)) \pmod{2^{256}}$)

       - Çarpanı uygulayın:

         $H_{\text{prev}} = (H_i \times m_i) \pmod{2^{256}}$

Zincirin son çıktısı olan H_{prev} (veya Sürüm 11'de $H_0 \times m_0$) değeri nihai blok özeti (block hash) olarak kabul edilir.

2.3.2. Aralarında Asal Çarpanın Matematiksel Özellikleri

Ara hash çıktılarının m_i değeri ile 2^{256} modunda çarpılması matematiksel olarak tam bir doğrusallık ve tutarlılık sunar. Modüler aritmetikte, bir m elemanının K moduna göre çarpımsal tersinin (multiplicative inverse) bulunabilmesi için $\gcd(m, K) = 1$ olmalıdır. 2^{256} modundaki tamsayılar grubu ($\mathbb{Z}_{2^{256}}$) için modül 2'nin bir kuvvetidir. Dolayısıyla, seçilen her tek tamsayı m_i, 2^{256} değeri ile aralarında asaldır:

$\gcd(m_i, 2^{256}) = 1$

Bu aralarında asallık ilişkisi, tanımlanan $f(x) = x \cdot m_i \pmod{2^{256}}$ fonksiyonunun bir bijeksiyon (birebir ve örten eşleme) olmasını garanti eder. Bu sayede:

- **Entropi Kaybı Olmaz**: Çarpım işlemi hash fonksiyonunun tüm entropisini korur; iki farklı girdi değeri aynı çıktı değerine eşlenemez.
- **Yozlaşma Engellenir**: Ara durumların sıfıra ya da daha dar bir alt gruba çökmesi (collapse) engellenir, kriptografik zincirin matematiksel bütünlüğü korunur.
- **Değişme Özelliğinin Olmaması**: Algoritmaların ve çarpanların belirli bir sıra ile uygulanması, sıralama değiştirme (order-swapping) saldırılarını imkansız kılar.

2.4. Korum Direnci ve Yer Tutucu Güvenliği

Tüm onaylayıcıların %100 katılımını zorunlu kılan mutabakat yapıları, tek bir düğümün çevrimdışı olması durumunda ağın durmasına neden olan liveness zafiyetine sahiptir. KristaTech, bu sorunu aşmak amacıyla **Korum Dirençli Yanıt Mekanizması** kullanır.

2.4.1. Yer Tutucu (Placeholder) Mekanizması

Geçerli çözüm sayısı `GetAdamThreshold(nHeight)` ile belirlenen asgari eşik değerini (bootstrap `< nAdamThresholdBootstrapLimit`, yani Mainnet'te 50.000, Testnet'te 100.000, Regtest'te 1.000. bloğun altında 5, sonrasında Mainnet/Regtest'te 7 ve Testnet'te 3) karşıladığı sürece blok şablonu başarıyla oluşturulur. Çözümünü zamanında ulaştıramayan onaylayıcıların `vAdamSolutions` dizisindeki yerleri, Koordinatör tarafından boş bayt vektörleri ile doldurulur:

vAdamSolutions[i] = std::vector<unsigned char>()

Bu sayede `vAdamMiners` ve `vAdamSolutions` arasındaki konumsal eşleşme korunur.

2.4.2. Doğrulama Mantığı

Doğrulama yapan düğümler, `CheckBlock()` fonksiyonunda şu adımları izler:

1. `vAdamSolutions` boyutunun `vAdamMiners` boyutu ile eşleştiğini doğrular.
2. `vAdamSolutions` elemanları üzerinde döngü çalıştırır. Boş vektörler yer tutucu olarak işaretlenip doğrulamadan muaf tutulur. Boş olmayan çözümler için kısmi iş kanıtı özetini hesaplar, imza ve zorluk derecesini denetler:
   - **Sürüm 11**:

     PuzzleHash = CalculateAdamPuzzleHash(algoIndex, Challenge)

burada $\text{algoIndex} = \text{Hash}(\text{Seed}_H \| \text{MinerPubKey}_i) \pmod{18}$ ve $\text{Challenge} = \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$ şeklindedir.

- **Sürüm 12**:

  $\text{PuzzleHash} = \text{algo1}((\text{minerIdx} + 1) \times \text{algo2}((\text{minerIdx} + 1) \times \text{algo3}(\text{Challenge}))) \pmod{2^{256}}$

  burada algo1, algo2 ve algo3 algoritmaları $\text{GetAdam3PermutationAlgos}(\text{hashPrevBlock}, \text{MinerPubKey}_i)$ ile elde edilir ve $\text{Challenge} = \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$ şeklindedir.

3. Başarıyla doğrulanan boş olmayan toplam çözüm sayısının T değerine eşit veya büyük olduğunu onaylar.

2.4.3. Güvenlik İspatları, Yedek Koltuk Devri ve Zamana Bağlı Eşik Gevşetme

- **Eşik Atlama Koruması**: Boş yer tutucuların imza veya zorluk doğrulama adımlarını geçmesi matematiksel olarak imkansızdır. Doğrulama motoru bunları geçersiz kanıt olarak değerlendirir. Ağın termodinamik iş yapma zorunluluğu korunur, çünkü en az T adet onaylayıcının geçerli ve yüksek zorlukta kanıt sunması şarttır.
- **Eşit Emek, Eşit Kazanç ve %50 Otomatik Protokol Yakımı**: **Eşit Emek, Eşit Kazanç** ilkesi uyarınca, çözüm göndermeyen yer tutucu koltuklar (`vAdamSolutions[i].empty()`) **%0** katılımcı ödülü alırken, bulmacasını çözen her onaylayıcı koltuğu (birincil veya $t \geq 60$ sn anında devreye giren aynı kulvar yedek onaylayıcısı) %40'lık Katılımcı Onaylayıcı Havuzundan (1 Coin-Lock koltuğu için %8 ve 3 Açık PoW koltuğu için koltuk başına %4 olmak üzere toplam %20; 7 Masternode onaylayıcı koltuğu için koltuk başına ~%2.857 olmak üzere %20) kendi kulvar payını alır. Çözülmeyen koltukların sahipsiz kalan payları (`nUnclaimedPartSplit`), **%50 Otomatik Protokol Yakımına (`nExpectedBurn = nUnclaimedPartSplit / 2`)** (dolaşımdaki `nMoneySupply` arzından saf deflasyon olarak kalıcı biçimde düşülür; kişisel PoB staking ağırlığına eklenmez) ve **%50 Geliştirici Hazinesine (`nExpectedTreasury = nTreasurySplit + (nUnclaimedPartSplit - nExpectedBurn)`)** bölüştürülür. Sessiz bir birincil koltuk meşru bir yedek onaylayıcı ile kurtarıldığında Koordinatör %15'lik blok üreticisi ödülünü kesintisiz olarak alır; birincil koltuklar ise birincil Koordinatörün münhasırlık penceresinin $t = 45$ sn'de sona ermesi, deterministik aynı kulvar yedek sıra düzeni ve aktif LLMQ'nun (`qblockprop`) yerel önbellekte geçerli çözümü bulunan birincil bir koltuğu değiştiren blok önerilerini veto etmesi sayesinde oportünistik ikamelere karşı korunur.
- **Yedek Doğrulayıcı Koltuk Devri ($t \geq 60$ sn) ve Zamana Bağlı Eşik Gevşetme ($t \geq 120$ sn)**: `nTimeElapsed >= 60 sn` sonrasında sessiz kalan birincil koltuklar, aynı kulvardaki (`MN -> MN`, `Coin-Lock -> Coin-Lock`, `Open PoW -> Open PoW`) sıradaki yedek adaylarla değiştirilebilir (`ResolveAdamCommitteeForTime` / `ValidateAdamCommitteeForTime`, reorg tutarlılığı için bloğun kendi `CCoinsViewCache* pView` görünümüne göre doğrulanır). Komite buna rağmen eşiğe ulaşamazsa, `GetAdamEffectiveThreshold(nHeight, nTimeElapsed, nActiveMasternodes)` fonksiyonu $t = 120$ sn 'den itibaren her `60 sn`'de bir eşiği `1` düşürerek asgari `ADAM_MIN_RELAXED_THRESHOLD = 5` değerine (`> 4` MN-dışı koltuk; bootstrap sonrasında veya aktif MN ≥ 7 iken en az 1 Aktif Masternode katılımını zorunlu kılar) kadar gevşetir.

---

3. Akıllı Sözleşme Dili: MESCAL

MESCAL (Minimalistically Envisioned Smart Contract Assembling Language), akıllı sözleşmelerin tanımlanması ve serileştirilmesi için geliştirilmiş JSON tabanlı bildirimsel (declarative) bir dildir.

3.1. Tasarım Felsefesi

Sadelik ve güvenlik odaklı bir yapıda tasarlanan MESCAL, doğrudan yığın tabanlı Bitcoin Script (`CScript`) bayt koduna derlenen bildirimsel bir paradigma benimser.

1. **Güvenlik**: Sadece blokzincir işlem kodlarıyla (opcodes) sınırlı, deterministik bir komut seti sunduğundan yeniden giriş (re-entrancy) açıklarını tamamen ortadan kaldırır.
2. **Gazsız Determinizm**: Döngü (loop) komutları barındırmadığından, sözleşmelerin çalışma süresi doğrusal ve öngörülebilirdir. Bu sayede karmaşık gaz tahmin mekanizmaları gereksiz kalır.
3. **Erişilebilirlik**: Sözleşmeler yapılandırılmış JSON dosyaları halinde tanımlandığından, istemci tarafındaki uygulamalar ve cüzdanlar tarafından kolayca okunabilir ve oluşturulabilir.

3.2. Gramer ve Resmi Sentaks (Syntax)

MESCAL akıllı sözleşmeleri belirli bir dil bilgisi (grammar) yapısına dayanır. Backus-Naur Formu (BNF) ile yazılmış sentaks şu şekildedir:

```
<contract_file>      ::= "{" <declaration_list> "," <contract_def> "," <active_field> "}"
<declaration_list>   ::= <basic_declaration> | <condition_declaration> | <declaration_list>
"," <declaration_list>
<basic_declaration>  ::= "\"basic\":" "{" <basic_definitions> "}"
<basic_definitions>  ::= <basic_entry> | <basic_definitions> "," <basic_entry>
<basic_entry>        ::= "\"" <identifier> "\":" "{" <role_def> "," <inputs_def> "}"
<role_def>           ::= "\"role\":" <opcode_string>
<inputs_def>         ::= "\"inputs\":" "[" <input_list> "]"
<input_list>         ::= <input_entry> | <input_list> "," <input_entry>
<input_entry>        ::= "{" "\"type\":" <type_string> "," "\"value\":" <value_string> "}"

<condition_declaration> ::= "\"condition\":" "{" <condition_definitions> "}"
<condition_definitions> ::= <condition_entry> | <condition_definitions> "," <condition_entry>
<condition_entry>       ::= "\"" <identifier> "\":" "{" "\"role\":" "\"if-condition\"" ","
<exprs_def> "," <true_path> "," <false_path> "}"

<contract_def>       ::= "\"contract\":" "{" "\"" <identifier> "\":" "{" "\"actions\":" "["
<action_list> "]" "}" "}"
<action_list>        ::= <action_entry> | <action_list> "," <action_entry>
<action_entry>       ::= "{" "\"type\":" <type_string> "," "\"name\":" <value_string> "}"
<active_field>       ::= "\"active_contract\":" "\"" <identifier> "\""
```

3.3. Komut Haritalama ve Teknik Detaylar

Derleyici, JSON bileşenlerini şu ikili işlemlere dönüştürür:

- **`hash160`** : Girdi verisini yığına iter ve SHA256 işlemini RIPEMD160 ile birleştirerek yürütür.
  - *CScript Karşılığı*: `<input> OP_HASH160`
- **`check-signature-verification`** : Bir açık anahtarı yığına iter ve kriptografik imza doğrulaması yapar.
  - *CScript Karşılığı*: `<pubkey> OP_CHECKSIGVERIFY`
- **`equalverify-checksig`** : Standart P2PKH doğrulaması yürütür.
  - *CScript Karşılığı*: `OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`

- **`multi-signature`**: Çoklu imza değerlendirmesi yapar.
  - *CScript Karşılığı*: `<m> <pubkeys...> <n> OP_CHECKMULTISIG`
- **`lock-time`**: Zaman veya blok yüksekliği kısıtlamalarını denetler.
  - *CScript Karşılığı*: `<locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP`
- **`number`**, **`add`**, **`sub`**, **`bool-and`**, **`bool-or`**, **`num-equal`**, **`greater-than`**, **`drop`**: Yığın aritmetik ve mantıksal temel işlemleri (`<int64_t>`, `OP_ADD`, `OP_SUB`, `OP_BOOLAND`, `OP_BOOLOR`, `OP_NUMEQUAL`, `OP_GREATERTHAN`, `OP_DROP`).
- **`coin-lock-miner`** & **`pow-miner`**: Madenci kimliğini, isteğe bağlı PoW zorluk/nonce kanıtını, CLTV zaman kilidini ve P2PKH harcama kurallarını içeren özel ADAM madenci kayıt betikleri.

3.4. Çalışma Güvenliği İspatı

C, sonlu sayıda yığın komutundan oluşan derlenmiş bir MESCAL sözleşmesi olsun: $I_1, I_2, \ldots, I_k$.

1. **Döngüsüz Çalışma**: Dil bilgisi kurallarında döngü (loop) ya da atlama (jump) komutları yer almaz. Dolayısıyla, kontrol akış grafiği (Control Flow Graph - CFG) yönlü asiklik bir grafiktir (Directed Acyclic Graph - DAG).
2. **Doğrusal Zaman Karmaşıklığı**: Sözleşmenin maksimum çalışma süresi komut sayısıyla doğrudan sınırlıdır:

   $E_{\max} = O(k)$

   Burada k değeri, JSON içindeki actions dizisinin boyutuna eşittir.
3. **Sonlanma Garantisi**: $E_{\max}$ değeri sonlu ve doğrusal olduğu için her MESCAL sözleşmesi kesin bir adımda sonlanmak zorundadır. Bu durum sonsuz döngü (infinite loop) ve kaynak tüketim saldırılarını tamamen engeller.
4. **Gazsız Çalışma**: Çalışma süresinin sonlu olduğu derleme aşamasında doğrulanabildiği için ağda karmaşık gaz hesaplama mekanizmaları barındırmaya gerek duyulmaz.

3.5. Örnek Şablon: Ölü Adam Anahtarı (Miras)

Belirli bir süre boyunca sahibinin anahtarı kullanılmadığında fonların otomatik olarak varise aktarılmasını sağlar, sahibi ise fonlara her an erişebilir:

- *CScript Karşılığı*:
  `<heir-pubkey> OP_CHECKSIGVERIFY OP_IF <expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_ELSE <owner-pubkey> OP_CHECKSIGVERIFY OP_ENDIF`

```
{
  "basic": {
    "Heir-Sig": {
      "role": "check-signature-verification",
      "inputs": [{ "name": "Pubkey", "type": "pubkey", "value": "02ee1fb8..." }]
    },
    "Lock-Time": {
      "role": "lock-time",
      "inputs": [{ "name": "Lock-Until", "type": "height", "value": 1780718400 }]
    },
    "Owner-Sig": {
      "role": "check-signature-verification",
      "inputs": [{ "name": "Pubkey", "type": "pubkey", "value": "03ee1fb8..." }]
    }
  },
  "condition": {
    "Inheritance-Condition": {
      "role": "if-condition",
      "expressions": [{ "type": "basic", "name": "Heir-Sig" }],
      "true": [{ "type": "basic", "name": "Lock-Time" }],
      "false": [{ "type": "basic", "name": "Owner-Sig" }]
    }
  },
  "contract": {
    "Inheritance-Switch": {
      "actions": [{ "type": "condition", "name": "Inheritance-Condition" }]
    }
  },
  "active_contract": "Inheritance-Switch"
}
```

3.6. Taproot (P2TR) Script-Path Entegrasyonu

MESCAL sözleşmeleri, yerleşik `compilemescaltotaproot` RPC komutu kullanılarak doğrudan Taproot (P2TR) script-path harcama koşullarına derlenebilir. Bu sayede geliştiriciler sözleşmeleri Bech32m biçimindeki P2TR adreslerine gömebilir, bu da ağda hem gizliliği hem de işlem boyutu verimliliğini artırır.

- **Derleme (Compilation)**: Derleme süreci, sözleşme JSON verisini ve bir `internal_pubkey` genel anahtarını alarak bir betik ağacı oluşturur. Çıktı olarak P2TR `address` adresini, `scriptPubKey` kilit betiğini, `leafScript` bayt kodunu ve 33 baytlık `controlBlock` verisini üretir.
- **Yürütüm (Execution)**: Çıktıyı harcamak için harcama işleminin tanık (witness) verisinde sırasıyla çalışma argümanları, `leafScript` ve `controlBlock` yer almalıdır (scriptSig alanına eklenir). Blokzincir üzerinde yalnızca fiilen yürütülen yaprak betik (leaf script) açığa çıkarılacağı için, kullanılmayan diğer koşul dalları (örneğin kurtarma veya zaman kilidi yolları) tamamen gizli kalır.

---

4. Ekosistem Tokenomisi

KristaTech, güvenlik teşviklerini korurken uzun vadeli arz kıtlığını destekleyen deflasyonist bir emisyon modeli uygular.

4.1. Temel Emisyon Parametreleri

- **Maksimum Arz Limit (Hard Cap)**: **210,000,000 KRISTA**

- **Ön Madencilik (Premine)**: **0 KRISTA** (tamamen adil dağıtım)
- **Hedef Blok Süresi**: 30 saniye (yılda yaklaşık 1.051.200 blok)
- **Teminat Gereksinimi**: 1. bloktan itibaren sabit **4,200 KRISTA**
- **Kademeli Bootstrap Aşaması (2 - 9,999. bloklar)**: Blok 2–2,200 arası **40 KRISTA**, Blok 2,201–5,000 arası **30 KRISTA**, Blok 5,001–9,999 arası **20 KRISTA**. Bu aşamada toplam **271,940 KRISTA** (~arzın %0.13'ü) üretilir. Bu sayede hem Model D ve LLMQ korumlarının kurulabilmesi için gereken aktif Masternode kurulumuna yetecek likidite sağlanır hem de erken dönem enflasyon baskısı engellenir.
- **Başlangıç Blok Ödülü (10,000. blok itibarıyla)**: **15 KRISTA**

4.2. Emisyon Azalma Modeli (Decay)

Ağın Bitcoin'deki gibi sert 4 yıllık yarılanma şoklarından kaçınması amacıyla, blok ödülleri her 259,200 blokta bir **%1.9 oranında azaltılır**. Hedef blok süresinin 30 saniye olması nedeniyle, bu 259.200 blokluk süreç takvim zamanı olarak yaklaşık 90 güne (üç ayda bir) denk gelmektedir. P periyodundaki blok ödülü şu şekilde hesaplanır:

$$\text{Reward}(P) = 15.0 \times (0.981)^{P}$$

Burada:

$$P = \left\lfloor \frac{\text{Height} - 10000}{259200} \right\rfloor$$

4.2.1. Maksimum Arz Limiti ve Boşluk Rezervinin Matematiksel İspatı

Emisyon modelinin hiçbir zaman 210M KRISTA sınırını aşmayacağını kanıtlamak için, toplam arz değerini bootstrap emisyonu ile periyotların geometrik serisi toplamının birleşimi olarak yazabiliriz. Maksimum dolaşım arzı $S_{\max}$ olsun:

$$S_{\max} = S_{\text{bootstrap}} + \sum_{P=0}^{\infty} \left(B_{\text{blocks}} \times R_0 \times (1-d)^{P} \right)$$

Burada:

- $S_{\text{bootstrap}} = 271,940$ KRISTA (2. blok ile 9,999. blok arasındaki kademeli üretim)
- $B_{\text{blocks}} = 259,200$ (azalma periyodundaki blok sayısı)
- $R_0 = 15.0$ KRISTA (decay başlangıç ödülü)
- $d = 0.019$ (decay oranı %1.9, çarpan değeri $1 - d = 0.981$)

$0 < (1 - d) < 1$ koşulu sağlandığı için sonsuz terimli geometrik seri yakınsar:

$$\sum_{P=0}^{\infty} (0.981)^{P} = \frac{1}{1 - 0.981} = \frac{1}{0.019} \approx 52.631579$$

Bu değerleri yerine yerleştirdiğimizde:

$$S_{\max} = 271,940 + 259,200 \times 15.0 \times \frac{1}{0.019}$$
$$S_{\max} = 271,940 + 3,888,000 \times 52.631579$$
$$S_{\max} = 271,940 + 204,631,579 \approx 204,903,519 \text{ KRISTA}$$

Maksimum arz limiti $S_{\max}$ ile 210M hard cap sınırı arasındaki fark **Boşluk Rezervini** (G_{reserve}) oluşturur:

$G_{\text{reserve}} = 210,000,000 - 204,903,519 = 5,096,481$ KRISTA

Bu **5,096,481 KRISTA (%2.43) Boşluk Rezervi**, blok ödüllerinin onlarca yıl boyunca aniden kesilmeden yumuşak bir şekilde sıfıra yaklaşmasını sağlar. Bu sayede ağın güvenlik bütçesi zamanla madencilik ödülünden işlem ücreti (fee) modeline sorunsuz olarak aktarılır.

```
Arz Doygunluk Grafiği:
[0] (Genesis) -> [18.99M] (1. Yıl) -> [32.72M] (2. Yıl) -> [68.12M] (5. Yıl) -> [111.71M] (10.
Yıl) -> [204.90M] (Asimptot)
```

4.3. Model D İşbirlikçi Blok Ödülü Paylaşımı

Blok ödülleri, hem işlem onaylayıcılarını hem de ağ altyapı sağlayıcılarını teşvik edecek şekilde bölünmüştür.

```
+-------------------------------------------------------------------------+
|                        Model D Blok Ödülü (%100)                        |
|             (Mainnet üzerinde 2.200+ bloklardan itibaren)               |
+-----------------------------------+-------------------------------------+
|       Masternode Havuzu (%45)     |       Onaylayıcı Havuzu (%55)       |
+-----------------+-----------------+-----------------+-----------------+
|    Pasif MN     |   LLMQ Aktif    |    Önerici /    |    Katılımcı    |
|  Sırası (%35)   |  Korumu (%10)   |  Kazanan (%15)  | Onaylayıcı (%40)|
+-----------------+-----------------+-----------------+-----------------+
```

- **İlk Aşama (2 - 2,199. bloklar)**: Masternode'lar henüz kurulamadığından ödülün %100'ü madenci/staker tarafına ödenir.
- **Korum Birikimi (2,000 - 2,199. bloklar)**: LLMQ korumları aktifleşir ve düğümler Masternode kurmaya teşvik edilir.
- **Model D Aktivasyonu (2,200. blok ve sonrası)**: Aşağıdaki dağıtım oranları zorunlu olarak uygulanır:
  1. **Pasif Masternode Payı (%35)**: Sıradaki Masternode'a ödenir (teminat kilitlenmesini teşvik eder).
  2. **LLMQ Korum Aktif Payı (%10)**: PoBLS biletlerini doğrulayan ve imzalayan aktif korum üyeleri arasında eşit olarak bölüştürülür.
  3. **Blok Üretici Payı (%15)**: Bloğu önermeye hak kazanan kazanan onaylayıcıya / Koordinatöre (PoW bloğu ise) veya paydaşa (PoS bloğu ise) kesintisiz olarak ödenir. İşlem ücretlerinin (fees) tamamı da blok üreticisi adresine gider.
  4. **Katılımcı Onaylayıcı Payı (%40 — 7 MN Koltuğu %20 + 4 Bağımsız Madenci Koltuğu %20)**: Çözüm üreten (`!vAdamSolutions[i].empty()`) komite koltukları arasında 7 Masternode onaylayıcı koltuğuna toplam **%20** (koltuk başına `~%2,857`) ve 4 bağımsız madenci koltuğuna (`1 Coin-Lock + 3 Open PoW`) toplam **%20** (1 Coin-Lock koltuğu için **%8,00** ve 3 Açık PoW koltuğu için koltuk başına **%4,00**) olarak dağıtılır. Boş kalan bağımsız madenci koltuğuna Masternode taşması (spillover) durumunda ilgili Masternode standart `~%2,857` payını alır ve aradaki fark (`%8,00` Coin-Lock koltuğu için `~%5,143` veya `%4,00` Açık PoW koltuğu için `~%1,143`) ile çözülmeyen/boş koltukların payları (`nUnclaimedPartSplit`) **%50 Otomatik Protokol Yakımına** ve **%50 Geliştirici Hazinesine** bölüştürülür.

4.3.1. Hibrid Bloklarda Ödül Dinamikleri (PoW vs. PoS)

Blok türüne göre dağıtımlar şu şekilde esner:

- **PoW Blokları**: %55'lik aktif çalışma payı ADAM havuzuna ödenir (%15 koordinatöre, %40 ise çözüm üreten komite koltuklarına [%20 7 MN + %20 4 bağımsız madenci] bölüştürülür; çözülmeyen koltuk paylarının %50'si Otomatik Protokol Yakımına, %50'si Geliştirici Hazinesine aktarılır).
- **PoS Blokları**: Stakleyen cüzdan %15'lik üretici payını alır. %40'lık onaylayıcı payı ise blok sürecinde geçerli çözüm sunan ADAM komite koltuklarına dağıtılarak madencilik altyapısının sürekliliği korunur.

4.4. Geliştirici ve Musluk Hazineleri

Protokolün sürdürülebilir fonlanması için ödüllerden doğrudan hazine kesintileri yapılır:

- **Geliştirici Hazinesi**: Blok 2'den itibaren tüm blok ödüllerinin **%7.0**'si doğrudan Geliştirici Fon Adresine (`KTeRC2Kj8hXcuzRohxPp2LtYFRJm2BceRoP`) aktarılır.
- **Musluk Fonu**: Blok 2 ile 50,000 arasında emisyonun **%0.7**'si Musluk Adresine (`KThn5Li5BnfGzbNJ92UDH7FL9CZR1VSRGEh`) yönlendirilir.

Bu kesintiler ham blok değerinden doğrudan düşülür. Örneğin, bootstrap 1. fazındaki (Blok 2–2,200) 40 KRISTA ödüllü bir blokta 2.8 KRISTA geliştiriciye, 0.28 KRISTA musluğa aktarılır ve kalan 36.92 KRISTA aktif konsensüs kurallarına göre paylaşılır.

---

5. Güvenlik ve Kriptografik Analiz

5.1. Sybil Saldırıları

Onaylayıcı seçimi veya bilet havuzunu ele geçirmeyi amaçlayan Sybil girişimleri yüksek ekonomik engellerle karşılaşır. **Çift Kulvarlı Kota Seçimi (`7 Aktif MN + 1 Coin-Lock + 3 Açık PoW`)** kapsamında, 11 komite koltuğunun **8'i** (`%72,7`) kilitli teminat gerektirir (**7 koltuk** için `4.200 KRISTA` Aktif Masternode teminatı ve **1 koltuk** için `250 KRISTA` Coin-Lock teminatı). Saldırgan sınırsız sayıda düşük maliyetli PoW-Lock kimliği oluştursa dahi en fazla **3 Açık PoW koltuğunu** ve **1 Coin-Lock koltuğunu** (toplam 4 koltuk) alabilir; bu sayı hem bootstrap sonrası `7` çoğunluk eşiğinin hem de zamana bağlı gevşetilmiş asgari `5` eşiğinin (`ADAM_MIN_RELAXED_THRESHOLD = 5`) altındadır. Dolayısıyla en az bir Aktif Masternode'un (`4.200 KRISTA`) onayı olmadan hiçbir blok üretilemez.

5.2. Ön Hesaplama ve Nothing-at-Stake Korumaları

- **Ön Hesaplama**: Geçici BLS anahtarları kalıcı kimlik anahtarından ve önceki blok özetinden deterministik olarak türetildiği için, onaylayıcılar bilet değerlerini önceden hesaplayarak piyangoyu manipüle edemez.
- **Nothing-at-Stake**: PoS ağlarında çift imzalama maliyetsizdir. KristaTech üzerinde rakip çatallarda (forks) oy kullanmak, en az T adet onaylayıcı koltuğunda fiziksel çoklu-algoritmalı PoW bulmacalarını çözmeyi gerektirir. Bu durum saldırgana reel bir donanım/enerji maliyeti yükler.

5.3. Çift Blok İmzası

Blok yüksekliği ≥ 200 (Cooperative PoS) olan bloklarda güvenlik iki farklı kriptografik imza ile kesin bir şekilde sağlanır:

> **ÖNEMLİ**
>
> **Tek İmzalı PoS Bloğu Yasaktır:** PoS blokları ağı tek imzalı bir konsensüs modeline geçirmez. Konsensüsün ele geçirilmesini veya doğrulamaların atlatılmasını önlemek amacıyla her PoS bloğunun her iki imzayı da taşıması zorunludur. Tek imzalı (geçici ya da kalıcı) bloklar, doğrulama yapan düğümler tarafından hiçbir koşulda kabul edilmez ve anında reddedilir.

1. **ADAM Koordinatör İmzası (`vAdamCoordinatorSig`)**: Seçilen onaylayıcı kümesinin ve oylama turlarının başarıyla tamamlandığını doğrular.
2. **Staker Blok İmzası (`vchBlockSig`)**: Blokta bulunan işlemlerin stakleyen UTXO özel anahtarıyla kilitlenmesini sağlar.

Doğrulama yapan tüm düğümler her iki imzanın da geçerliliğini şart koştuğu için ağ, geçmişi yeniden yazma veya mutabakatı ele geçirme girişimlerine karşı koruma altındadır.

---

6. Sonuç

KristaTech blokzincir protokolü, geleneksel ağların kısıtlamalarını aşan işbirlikçi bir konsensüs tasarımı sunmaktadır. Onaylayıcı seçimi (ADAM) ile blok önerme (PoBLS) süreçlerinin ayrılması sayesinde ağda önerici anonimliği sağlanmış, Sybil saldırıları engellenmiş ve hedefli DDoS tehditleri bertaraf edilmiştir. Korum dirençli yer tutucu mekanizması zincir canlılığını (liveness) korurken, bildirimsel MESCAL dili güvenli ve gaz ücreti karmaşasından uzak bir akıllı sözleşme ortamı sunar. %1.9 azalma oranına sahip 210 Milyonluk emisyon modeli ve Model D İşbirlikçi Paylaşımı ile desteklenen KristaTech; madenciler, paydaşlar ve masternode işletmecileri için dengeli bir teşvik yapısı kurarak sürdürülebilir bir blokzincir platformu sağlamaktadır.

---

7. Referanslar

1. Nakamoto, S. (2008). “Bitcoin: A Peer-to-Peer Electronic Cash System.”
2. Micali, S., Rabin, M., & Vadhan, S. (1999). “Verifiable Random Functions.” *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*.
3. Wood, G. (2014). “Ethereum: A Secure Decentralised Generalised Transaction Ledger.”
4. Boneh, D., Gentry, C., Lynn, B., & Shacham, H. (2003). “Aggregate and Verifiable Signatures from Bilinear Maps.” *Journal of Cryptology*.
5. Maymounkov, P., & Mazieres, D. (2002). “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric.” *International Workshop on Peer-to-Peer Systems*.